Review - Midterm

Exercise 1

Indicate whether the first function of each of the following pairs has a smaller, same, or larger order of growth (to within a constant multiple) than the second function.

- Indicate whether the first function of each of the following pairs has a smaller, same, or larger order of growth (to within a constant multiple) than the second function.
  1. $n(n + 1)$ and $2000n^2$

     Both are **$\Theta(n^2)$**.
     Same order of growth.

  2. $100n^2$ and $0.01n^3$

     First is **$\Theta(n^2)$**, Second is **$\Theta(n^3)$** n^3 grows faster than n^2, so, First function has **smaller** order of growth.

  3. $\log_2 n$ and $\ln n$

     $$\log_2 n = \frac{\ln n}{\ln 2}$$

     They differ only by a constant factor, so same order of growth (both $\Theta(\log n)$).

  4. 2^{n-1} and 2^n

     $$2^{n-1} = \frac{1}{2} \cdot 2^n$$

     Only a constant factor difference, so same order of growth (both $\Theta(2^n)$).

  5. $(n-1)!$ and $n!$

     $$n! = n \cdot (n-1)!$$

     The second function is larger by a factor of n, so the first function has **smaller** order of growth.

Exercise 2

**Use the informal definitions of O, Ω ,Θ band to determine whether the following assertions are true or false.

a. $n(n+1)/2 \in O(n^3)$ **b.** $n(n+1)/2 \in O(n^2)$
c. $n(n+1)/2 \in \Theta(n^3)$ **d.** $n(n+1)/2 \in \Omega(n)$

a: True. Big O notation says that order of growth should be *smaller* or *equal*
b: True. Same reason as above
c: False. Theta notation states first function should have same order of growth as the second one.
d: True. Quadratic has higher order of growth. Omega is greater than or equal.

or false.

T **a.** $n(n+1)/2 \in O(n^3)$ **b.** $n(n+1)/2 \in O(n^2)$ T

F **c.** $n(n+1)/2 \in \Theta(n^3)$ **d.** $n(n+1)/2 \in \Omega(n)$ T

Exercise 3

List the following functions according to their order of growth from the lowest to the highest

$$(n-2)!,\quad 5\lg(n+100)^{10},\quad 2^{2n},\quad 0.001n^4+3n^3+1,\quad \ln^2 n,\quad \sqrt[3]{n},\quad 3^n.$$

$(n-2)!,\ 5\lg(n+100)^{10},\ 2^{2n},\ 0.001n^4+3n^3+1,\ \ln^2 n,\ \sqrt[3]{n},\ 3^n.$

1. 5lg(n + 100)^10
2. ln^2 n
3. cbrt(n)
4. 0.001n^4 + 3n^3
5. 3^n
6. 2^2n
7. (n - 2)!

- List the following functions according to their order of growth from the lowest to the highest:

7 1 6 4 2 3 5

$$(n-2)!,\quad 5\lg(n+100)^{10},\quad 2^{2n},\quad 0.001n^4+3n^3+1,\quad \ln^2 n,\quad \sqrt[3]{n},\quad 3^n.$$

$$\boxed{50\lg(n+100) < \ln^2 n < \sqrt[3]{n} < n^4 < 3^n < 4^n < (n-2)!}$$

Exercise 4

For each of the following algorithms, indicate
(i) a natural size metric for its inputs,
(ii) its basic operation
(iii) whether the basic operation count can be different for inputs of the same size:

1. **computing the sum of n numbers**
    1. n - the count of the numbers
    2. basic operation is the sum or addition
    3. it will be the same for all the inputs of size n
2. **computing n!**
    1. n the integer whos factorial is being computed
    2. Basic operation: multiplication
    3. The count only depends on n.
3. **finding the largest element in a list of n number**
    1. n, the number of elements in the list
    2. Basic op: Comparison
    3. The operation count depends only on n, not the input values

For each of the following algorithms, indicate(i) a natural size metric for its inputs, (ii) its basic operation, and (iii) whether the basic operation count can be different for inputs of the same size:

1. computing the sum of n numbers
    - n, the number of elements in the list.
    - Basic operation: Addition
    - The operation count depends only on n, not on input values.
1. computing n!
    - n(the integer whose factorial is being computed).
    - Basic operation: Multiplication.
    - The count depends only on n.
1. finding the largest element in a list of n numbers
    - n, the number of elements in the list.
    - Basic operation: Comparison
    - The operation count depends only on n, not on the input values.

Exercise 5

For each of the following algorithms, indicate whether it is:
• Stable?
• In-place?
Algorithms:
• Insertion Sort

- **Selection Sort**
- **Bubble Sort**

1. **Insertion sort:**
    1. Stable
    2. Yes, In-place
2. **Selection sort:**
    1. Not stable
    2. Yes, In-place
3. **Bubble sort**
    1. Stable
    2. Yes, In-place

Exercise 6

- Consider the following algorithm.

ALGORITHM *Mystery*(n)

```
//Input: A nonnegative integer n
S ← 0
for i ← 1 to n do
    S ← S + i * i
return S
```

1. What is its basic operation?
2. How many times is the basic operation executed?
3. What is the efficiency class of this algorithm?

1. Basic op is addition and multiplication. No need to choose since addition will tell us how many types multiplication will run, but if we have to we choose, then we pick multiplication
2. $M(n) = \sum_{i=1}^{n} 1 = n$
3. This is O(n)

Exercise 7

- Consider the following algorithm.

```
ALGORITHM  Secret(A[0..n − 1])
    //Input: An array A[0..n − 1] of n real numbers
    minval ← A[0]; maxval ← A[0]
    for i ← 1 to n − 1 do
        if A[i] < minval
            minval ← A[i]
        if A[i] > maxval
            maxval ← A[i]
    return maxval − minval
```

1. What is its basic operation?
2. How many times is the basic operation executed?
3. What is the efficiency class of this algorithm?

1. Basic op is comparison
2. $\boxed{C(n) = 2\sum_{i=1}^{n-1} 1 = 2(n-1)}$
3. O(n)

Exercise 8

- Consider the definition-based algorithm for adding two n × n matrices.
- What is its basic operation?
- How many times is it performed as a function of the matrix order n?
- As a function of the total number of elements in the input matrices?

```
Algorithm MatrixAddition(A[0..n-1], B[0..n-1])
  Input: Matrices A and B of size n × n
  Output: Matrix C of size n × n such that C[i][j] = A[i][j] + B[i][j]

  for i ← 0 to n - 1 do
    for j ← 0 to n - 1 do
      C[i][j] ← A[i][j] + B[i][j]

  return C
```

1. Basic operation is addition

2. $\boxed{\mathbf{A}(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathbf{1} = n \cdot n = n^2}$

3. O(n^2)

Exercise 9

- Given the algorithm below to finding the distance between the two closest elements in an array of numbers, answer the following questions.
  1. What's the basic operation?
  2. Write a summation that represents the number of times the basic operation is executed.
  3. Solve the summation and find the function which represents the number of times the basic operation is executed. Show your work.
  4. Classify the function above into one of the standard complexity classes using Θ-notation.

```
ALGORITHM  MinDistance(A[0..n − 1])
    //Input: Array A[0..n − 1] of numbers
    //Output: Minimum distance between two of its elements
    dmin ← ∞
        for i ← 0 to n − 1 do
            for j ← 0 to n − 1 do
                if i ≠ j and |A[i] − A[j]| < dmin
                dmin ← |A[i] − A[j]|
    return dmin
```

1. Basic op is comparison

2. $\mathbf{C}(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathbf{1}$

3. $\boxed{\mathbf{C}(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathbf{1} = n \cdot n = n^2}$

4. O(n^2)

Exercise 10

- Given the following algorithm, answer the following questions.
  1. What's the basic operation?
  2. Write a summation that represents the largest number of times the basic operation is executed.
  3. Solve the summation and find the function which represents the largest number of times the basic operation is executed. Show your work.
  4. Classify the function above into one of the standard complexity classes using Θ-notation.

```
ALGORITHM FindFirstDuplicate(arr, size)
// Input: An array of integers, and its size
// Output: The first duplicate element found in the array, or -1 if no duplicates.

i <-- 0
while i < size do
   j <-- i + 1
   while j < size do
      if arr[i] = arr[j] then
         return arr[i]
      j <-- j + 1
   i <-- i + 1

return -1  // No duplicates found
```

1. Basic op is comparison
2. $C(n) = \sum_{i=0}^{size-1} \sum_{j=i+1}^{size-1} 1$
3. $C(n) = \sum_{i=0}^{size-1} \sum_{j=i+1}^{size-1} 1$
4. O(n^2)

Exercise 11

- Consider the following algorithm for sorting an array: for each element, it counts how many elements are smaller than it and uses this count to place the element in its correct position in the sorted array. Analyze the efficiency of this algorithm.

```
ALGORITHM ComparisonCountingSort(A[0..n − 1])
    //Sorts an array by comparison counting
    //Input: Array A[0..n − 1] of orderable values
    //Output: Array S[0..n − 1] of A's elements sorted
    //    in nondecreasing order
    for i ← 0 to n − 1 do
        Count[i] ← 0
    for i ← 0 to n − 2 do
        for j ← i + 1 to n − 1 do
            if A[i] < A[j]
                Count[j] ← Count[j] + 1
            else Count[i] ← Count[i] + 1
    for i ← 0 to n − 1 do
        S[Count[i]] ← A[i]
    return S
```

O(n^2)

Exercise 12

- Consider the following recursive algorithm.

```
Algorithm ComputeX(n)
    Input: Integer n ≥ 1 (assumed power of 2)
    Output: Value of x(n)

    if n == 1 then
        return c  // base case
    else
        return ComputeX(n / 2) + n
```

- What's the basic operation?
- Write a recurrence relation that represents the number of times the basic operation is executed.
- Solve the recurrence relation using backward substitution and find the function which represents the number of times the basic operation is executed. Show your work.
- Classify the function above into one of the standard complexity classes using Θ-notation.

1. Basic op is addition - In recursive algos its found in the recursive call
2. A(n) = A(n / 2) + 1 -> A(1) = 0
3. Assume 2^k -> A(2^k) = A(2^k-1) + 1 -> A(2^0) = 0 ->
    1. A(2^k) = A(2^k-1) + 1
    2. sub. A(2^k - 1) = A(2^k - 2) + 1
    3. = A(2^k - 2) + 2
    4. = A(2^k - 3) + 3
    5. **A(2^k - i) + i**
    6. **A(2^k - k) + k**
    7. 0 + k -> k = logbase2n
4. O()

Exercise 13

- Consider the following recursive algorithm.

```
Algorithm MysteryRecursive(n):
    if n <= 1:
        return 1
    else:
        return MysteryRecursive(n-1) + MysteryRecursive(n-1)
```

- What's the basic operation?
- Write a recurrence relation that represents the number of times the basic operation is executed.
- Solve the recurrence relation using backward substitution and find the function which represents the number of times the basic operation is executed. Show your work.
- Classify the function above into one of the standard complexity classes using Θ-notation.

1. Basic op is addition

2. 2A(n - 1) + 1
3. 2^n

Exercise 14

- Solve the following recurrence relations.

a. $x(n) = x(n-1) + 5$ for $n > 1$, $x(1) = 0$
b. $x(n) = 3x(n-1)$ for $n > 1$, $x(1) = 4$
c. $x(n) = x(n-1) + n$ for $n > 0$, $x(0) = 0$
d. $x(n) = x(n/2) + n$ for $n > 1$, $x(1) = 1$ (solve for $n = 2^k$)
e. $x(n) = x(n/3) + 1$ for $n > 1$, $x(1) = 1$ (solve for $n = 3^k$)